Project - Low Level Design

on

Chat with Technology Documents (RAG)

Course Name: GEN AI

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
1.	Parthiv Prajapat	EN22EL301039
2.	Pranjal Mahawar	EN22CS301729
3.	Prateeksha Shukla	EN22CS301740
4.	Raj Patel	EN22CS301781
5.	Rishabh Dev Rajput	EN22CS301803

Group Name: 06D7

Project Number: GAI 29

Industry Mentor Name: Aashruti Shah

University Mentor Name: Dr. Hemlata Patel

Academic Year: 2025-2026

Table of Contents

Chapter 1	Introduction	3-4
	1.1. Scope of the Document	3
	1.2. Intended Audience	3
	1.3. System Overview	4
Chapter 2	System Design	5-9
	2.1. Application Design	5
	2.2. Process Flow	6
	2.3. Information Flow	7
	2.4. Components Design	7
	2.5. Key Design Considerations	8
	2.6. API Catalogue	9
Chapter 3	References	10

1. Introduction

Retrieval-Augmented Generation (RAG) is an advanced approach in Generative AI that combines information retrieval techniques with large language models (LLMs) to produce accurate and context-aware responses. Traditional language models rely solely on pre-trained knowledge, which may lead to outdated or hallucinated outputs. In contrast, RAG enhances reliability by retrieving relevant information from external data sources and grounding responses in that content.

This project focuses on designing and developing a complete RAG-based system for interacting with technology-related documents such as PDF reports, manuals, and technical documentation. The system allows users to upload documents, which are then processed through a structured pipeline involving text extraction, chunking, and vector embedding. These embeddings are stored in a vector database for efficient similarity-based retrieval.

1.1 Scope of the Document

This document outlines the design, development, and implementation of a Retrieval-Augmented Generation (RAG) system for technology document intelligence. It provides a comprehensive overview of the system architecture, components, and workflows involved in enabling intelligent document-based question answering.

The scope of this project includes:

Document Processing:
Handling various document formats (primarily PDFs), extracting text, and preprocessing it for further use.

Text Chunking and Embedding:
Dividing large documents into manageable chunks and converting them into vector representations using embedding models.

Vector Database Integration:
Storing and managing embeddings in a high-performance vector database to enable fast and relevant similarity searches.

Query Processing and Retrieval:
Accepting user queries and retrieving the most relevant document segments based on semantic similarity.

LLM Integration:
Using LLM APIs to generate accurate, context-aware responses grounded in retrieved document content.

User Interaction Interface:
Providing a simple interface (CLI or web-based) for users to upload documents and ask questions.

System Design and Architecture:
Defining the overall structure, including data flow, API interactions, and component communication.

Limitations:
The system is limited to the knowledge present in uploaded documents and does not generate answers beyond the provided content. Performance may vary depending on document quality and embedding effectiveness.

1.2 Intended Audience

This document is intended for individuals and groups involved in the design, development, evaluation, and usage of the Retrieval-Augmented Generation (RAG) system for technology document intelligence. The primary audience includes:

Students and Academic Evaluators:
Students working on Generative AI or related projects can use this document as a reference for understanding RAG system architecture and implementation. Faculty members and evaluators can review the design and approach used in the project.

Developers and Engineers:
Software developers, AI engineers, and data scientists who are interested in building document-based question-answering systems can use this document to understand system components such as text processing, embeddings, vector databases, and LLM integration.

Researchers in Generative AI:
Researchers exploring Retrieval-Augmented Generation, Natural Language Processing (NLP), and information retrieval can benefit from the system design and workflow described in this document.

Industry Professionals:
Professionals working in organizations that handle large volumes of technical documentation (e.g., IT companies, support teams, knowledge management teams) can use this system as a reference for implementing intelligent document search and Q&A solutions.

End Users:

Individuals who need to interact with technical documents—such as engineers, analysts, or support staff—can use the system to quickly retrieve accurate, context-based answers from uploaded documents.

This document is intended for software developers, system architects, project evaluators, and technical stakeholders who are involved in the development, review, or assessment of the system. It provides detailed technical insights into the implementation of Generative AI within an educational real estate application.

Additionally, this document is useful for students and researchers who want to understand how modern AI technologies such as LLMs and vector databases can be applied in real-world scenarios. It helps bridge the gap between theoretical concepts and practical implementation by explaining how different components interact within the system. The document is written in a structured and understandable manner so that both technical and semi-technical audiences can benefit from it.

1.3 System Overview

The proposed system is a Retrieval-Augmented Generation (RAG) based solution designed to enable intelligent interaction with technology documents such as PDFs, manuals, and reports. It combines document processing, semantic search, and large language model (LLM) capabilities to deliver accurate and context-aware responses to user queries.

The system follows a structured pipeline consisting of multiple interconnected components. Initially, users upload documents through the interface, which are then processed to extract raw text. This text is segmented into smaller chunks to improve retrieval efficiency and contextual relevance. Each chunk is transformed into vector embeddings using an embedding model, capturing the semantic meaning of the text.

These embeddings are stored in a vector database, which enables fast similarity-based retrieval. When a user submits a query, the system converts the query into an embedding and searches the database to find the most relevant document chunks. These retrieved chunks are then passed to an LLM via an API, which generates a response grounded strictly in the provided context.

The system ensures that all responses are based only on the uploaded documents, reducing the risk of incorrect or hallucinated answers. It is designed to be scalable, modular, and efficient, making it suitable for both academic and practical use cases.

2. System Design

2.1 Application Design

Application Design

The application is designed as a modular and scalable system that integrates document processing, semantic search, and Large Language Model (LLM) capabilities. It follows a layered architecture to ensure separation of concerns, maintainability, and efficient data flow between components.

The system is divided into three primary layers: Presentation Layer, Application Layer, and Data Layer.

1. Presentation Layer (User Interface)

This layer provides the interface through which users interact with the system. It can be implemented as a web application or a command-line interface (CLI).

Key Features:

Document upload functionality (PDF/manuals)

Query input interface

Display of generated responses

User-friendly interaction

Technologies (optional):

Web: HTML, CSS, JavaScript, Streamlit or Flask

CLI: Python-based terminal interface

2. Application Layer (Core Logic)

This is the core layer where all processing and decision-making occurs. It coordinates between the user interface and the data layer.

a. Document Processing Module

Extracts text from uploaded documents

Cleans and preprocesses text

Splits text into smaller chunks

b. Embedding Module

Converts text chunks into vector embeddings

Uses pre-trained embedding models (e.g., OpenAI embeddings)

c. Query Processing Module

Accepts user queries

Converts queries into embeddings for similarity comparison

d. Retrieval Module

Searches the vector database

Retrieves top-k relevant chunks based on similarity

e. LLM Integration Module

Sends retrieved chunks along with the query to the LLM API

Generates context-aware responses

Ensures responses are grounded in retrieved data only

3. Data Layer

This layer manages data storage and retrieval.

a. Vector Database

Stores embeddings of document chunks

Supports fast similarity search

Examples: FAISS, Pinecone, ChromaDB

b. Document Storage

Stores original uploaded documents (optional)

Maintains metadata such as file name, upload date, etc.

2.2 Process Flow

Process Flow

The Retrieval-Augmented Generation (RAG) system follows a systematic process to transform raw documents into meaningful, context-aware responses. The workflow is divided into two main phases: Document Processing Phase and Query Processing Phase.

1. Document Processing Phase

This phase is responsible for preparing the uploaded documents for efficient retrieval.

Step 1: Document Upload

The user uploads a document (PDF/manual) through the application interface.

Step 2: Text Extraction

The system extracts raw text from the uploaded document using a PDF parser.

Step 3: Text Preprocessing

The extracted text is cleaned by removing unnecessary characters, formatting issues, and noise.

Step 4: Text Chunking

The processed text is divided into smaller chunks to improve retrieval accuracy and performance.

Step 5: Embedding Generation

Each text chunk is converted into a vector embedding using a pre-trained embedding model.

Step 6: Storage in Vector Database

The generated embeddings are stored in a vector database for fast similarity search.

2. Query Processing Phase

This phase handles user queries and generates responses.

Step 1: User Query Input

The user submits a question through the interface.

Step 2: Query Embedding

The system converts the user query into a vector embedding.

Step 3: Similarity Search

The vector database is searched to find the most relevant document chunks based on similarity scores.

Step 4: Context Retrieval

Top relevant chunks are selected and combined to form the context.

Step 5: LLM Response Generation

The retrieved context and user query are sent to the LLM API.

The LLM generates a response strictly based on the provided context.

Step 6: Response Display

The final answer is displayed to the user through the interface.

3. Process Flow Diagram (Textual Representation)

```
[User Uploads Document]
        ↓
   [Text Extraction]
        ↓
   [Text Preprocessing]
        ↓
     [Chunking]
        ↓
  [Embedding Generation]
        ↓
  [Vector Database Storage]
        ↓
-----------------------------------
        ↓
   [User Enters Query]
        ↓
   [Query Embedding]
        ↓
   [Similarity Search]
        ↓
   [Context Retrieval]
        ↓
  [LLM Response Generation]
        ↓
```

[Display Response]

2.3 Information Flow

The Information Flow in the Retrieval-Augmented Generation (RAG) system describes how data moves between different components, from document ingestion to response generation. It ensures that the system processes inputs efficiently and produces accurate, context-aware outputs.

The flow of information is sequential and controlled, allowing each module to perform a specific function while maintaining data integrity and relevance.

2.4 Components Design

The RAG-based system is composed of several modular components, each responsible for a specific function in the overall workflow. This modular design ensures scalability, maintainability, and efficient processing of both documents and user queries.

[User Interface]

↓

[Document Processing] → [Chunking] → [Embedding] → [Vector DB]

↑

[User Query] → [Query Embedding] → [Retrieval Component]

↓

[LLM Integration]

↓

[Response Handling]

↓

[User Interface]

2.5 Key Design Considerations

Designing a Retrieval-Augmented Generation (RAG) system requires careful planning to ensure accuracy, efficiency, scalability, and reliability. The following key considerations were taken into account during the system design and development:

1. Accuracy and Context Relevance

The system must generate responses strictly based on retrieved document content.

High-quality embeddings and effective similarity search are essential to ensure relevant context retrieval.

Proper chunk size and overlap strategies are used to preserve meaning and avoid loss of context.

2. Minimizing Hallucination

The LLM is constrained through prompt engineering to use only the provided context.

Responses are grounded in retrieved document chunks to reduce incorrect or fabricated information.

3. Efficient Retrieval Performance

A high-performance vector database is used to enable fast similarity searches.

Optimization techniques such as indexing and top-k retrieval are applied to reduce latency.

4. Scalability

The system is designed to handle large volumes of documents and multiple users.

Modular architecture allows easy scaling of components such as the vector database and API services

2.6 API Catalogue

API Catalogue

The API Catalogue defines the set of application programming interfaces used in the Retrieval-Augmented Generation (RAG) system. These APIs enable communication between different components of the system, ensuring smooth data flow from document upload to response generation. Each API is designed to handle a specific functionality within the system.

1. Document Upload API

This API is responsible for handling the upload of documents into the system. It allows users to submit PDF files or technical documents through the interface. Once uploaded, the system assigns a unique identifier to each document, which is used for further processing.

2. Document Processing API

The Document Processing API extracts text from uploaded documents and prepares it for further operations. It performs preprocessing tasks such as cleaning, normalization, and formatting. The processed text is then divided into smaller chunks to improve retrieval efficiency.

3. Embedding Generation API

This API converts text chunks into vector embeddings using a pre-trained embedding model. These embeddings capture the semantic meaning of the text and are essential for similarity-based search operations within the system.

Vector Storage API

The Vector Storage API stores the generated embeddings into a vector database along with relevant metadata. This ensures that the system can efficiently retrieve document content during query processing.

5. Query API

The Query API handles user queries submitted through the interface. It acts as the central API that coordinates the retrieval and response generation processes. It ensures that user questions are properly processed and answered.

6. Retrieval API

This API is responsible for fetching the most relevant document chunks from the vector database. It uses similarity search techniques to identify content that closely matches the user query, ensuring accurate and context-aware retrieval.

7. LLM Response API

The LLM Response API interacts with the Large Language Model to generate final answers. It takes the user query along with the retrieved document context and produces a response that is strictly grounded in the provided information.

8. Metadata API

The Metadata API manages information related to uploaded documents, such as file names, document IDs, and timestamps. It helps in organizing and tracking documents within the system.

3. References

1. FastAPI Documentation. *FastAPI – Modern, Fast Web Framework for Building APIs with Python.*
   Available at: https://fastapi.tiangolo.com/
2. MongoDB Documentation. *MongoDB Official Documentation for NoSQL Database Management.*
   Available at: https://www.mongodb.com/docs/
3. Docker Inc. *Docker Documentation – Containerization and Deployment Platform.*
   Available at: https://docs.docker.com/
4. LangChain Documentation. *LangChain Framework for Building Applications with Large Language Models.*
   Available at: https://python.langchain.com/
5. OpenAI. *OpenAI API Documentation – Generative AI and LLM Usage.*
   Available at: https://platform.openai.com/docs/
6. FAISS Documentation. *Facebook AI Similarity Search (FAISS) for Vector Database Implementation.*
   Available at: https://faiss.ai/
7. Pinecone Documentation. *Pinecone Vector Database for Semantic Search and Retrieval.*
   Available at: https://www.pinecone.io/docs/
8. ChromaDB Documentation. *Chroma Vector Database for AI Applications.*
   Available at: https://docs.trychroma.com/
9. Redis Documentation. *Redis In-Memory Data Store for Caching and Performance Optimization.*
   Available at: https://redis.io/documentation/